cw1 very good

Report on Machine Learning for Computer Vision
Coursework 1 on Randomised Decision Forest

Xinyi Ren 01225077 Danni Huang 01222119
Electrical & Electronic Engineering
Imperial College London
xinyi.ren16@imperial.ac.uk
danni.huang16@imperial.ac.uk

I. INTRODUCTION

Random Forest (RF) is an efficient algorithm in machine learning that can achieve highly scalable training and even real-time testing when solving tasks like classification, regression and clustering.

As a classifier, Random Forest is more capable to handle multi-class problems comparing to other techniques like SVM, as well as high dimensional problems since no dimension reduction is required.

A RF simply consists of several decision trees. For each tree, randomness is injected to its training set and node optimization respectively to make the result random, thus the ensemble can effective avoid the overfitting of the decision tree algorithm and perform well on new unseen testing.

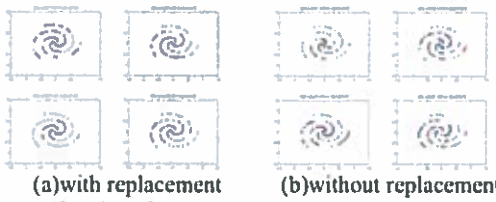
II. PROCEDURE

2.1 Training Decision Forest

Bagging (Boostrap Aggregating) is a technique to separate data to several subsets by randomly sampling N data points under uniform distribution from the original dataset for T times, where N is the size of each data subset and T donates the number of subsets.

The sampling is usually applied with replacement, where there is quite a chance that same points are chosen into the subset multiple times. If N is equal to the size of the original dataset and is a large number, each subset is expected to have a fraction of $1-\frac{1}{e}\approx 63.2\%$ of the unique data, while the rest shows repetition.

Fig.1(a) and (b) illustrate the four-subset bagging from the training data with replacement and without replacement respectively. 2 the with replacement case, we set the size of each subset equals to that of the original dataset, there are 4 averagely 95 unique data points in each subset. In terms of without replacement, we choose 75 out of 150 different data points in each subset. 5 ompared to without replacement, sampling with replacement gives relatively stable number of fraction of unique data and provide good randomness in training set.



(a)with replacement (b)without replacement

Fig.1 bagging results for four data subsets

In fig.1, the points in red, green, and blue color donate data with label 1, 2 and 3 respectively.

The split functions is the divider in nodes to decide the data points which child node to go, and in terms of different conditions, there are four type of split function could be chosen. In this paper, the split function we utilized in each type is shown below.

Linear:

$$h(v,\theta)=[x_i>k*x_j+b] \quad (1)$$

Non-linear:

$$h(v,\theta)=[x_2>a*x_1{}^2+b] \quad (2)$$

Axis-aligned:

$$h(v,\theta)=[x_i>\tau] \quad (3)$$

Two-pixel test:

$$h(v,\theta)=[x_i-x_j>\tau] \quad (4)$$

Where $i,j\in\{1,\dots,d\}$ *and* $i\neq j$. d represents the dimension of the data k, b, a are all random coefficients. τ is a randomly set threshold.

Evaluating the influence by the choice of weak learner, information gain is introduced to estimate the behavior on decision tree prediction.

Assume a dataset S with c represents the class labels and C donates the set of all classes, the entropy is given by

$$H(S)=-\sum_{c\in C}P(c)logP(c) \quad (5)$$

where $P(c)$ indicates the empirical distribution of each class.

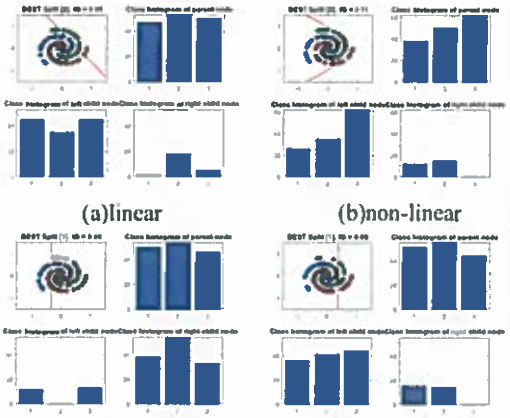
1 through the tree, children distributions would generally be purer than the parent one, then the entropy decrease and the information content increase.

The information gain is so that to measure the information content increase.

$$I(S,\theta)=H(S)-\sum_{t\in\{L,R\}}\frac{|s^t|}{|s|}H(s^t) \quad (6)$$

where S is the dataset reaching this node while s^L and s^R respectively donate the data subset that would go to the left and right child node.

For a chosen type of split function, the best randomness in node is realized by controlling 3 he number of split (i.e. the degree of randomness parameter ρ) to try to select the one who achieves the best information gain.



(a)linear (b)non-linear

(c)axis-aligned (d)two pixel test

Fig.2 best splitting result and class histograms of root

Summary of Comments on mlcv_cw1_very_good.pdf

Page: 1

Number: 1 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:42:36

Number: 2 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:41:50

Number: 3 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:43:14

Number: 4 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:42:13
very good

Number: 5 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:41:59

Number: 6 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:43:05
excellent!

node($\rho = 5$)

Given a 2D dataset, the best splitting results of the root node of an arbitrary subset bagging with different types of split function are given in fig.2, where the degree of randomness is settled at 5.

In this 2D case, the red line in the top left subgraph in each image is the visualization of the split function used to get the best information gain among the several times try (controlled by the degree of randomness). The split function gives a splitting threshold to the data set, data points are classified by the threshold to the left or right child node in terms of whether they are included in the threshold or not, then the distribution of the included data points with three different labels is stored in this node.

Tab.1 illustrates the averaged best information gain of several trials with four split functions and various degree of randomness.

ρ	The split function			
	Linear	Non-linear	axis-aligned	Two pixel
3	0.05	0.04	0.09	0.09
5	0.05	0.11	0.08	0.05
10	0.08	0.12	0.09	0.11
30	0.13	0.12	0.12	0.10

Tab.1 averaged best information gain of four split functions with different ρ

For the same degree of randomness, the classification results utilizing the four different split functions show no obvious difference.

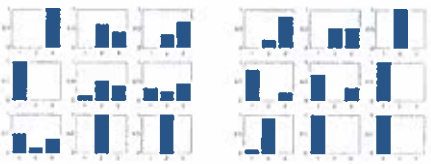
For a specific split function, the best information gain is seen a rise with the increasing of ρ by 10 and then becomes stable, which tells that the degree of randomness could optimizes the classification result in a range. Thus, we choose $\rho = 10$ in the follow-on tests, as it ensures the relatively good classification, and relatively save the computation time.

Trees grow with nodes recursively split until stopping criteria tell them to stop. Without the control of the tree depth and the number of data points in each split node, the tree would endlessly grow until data points are individually classified, and the training of classification becomes meaningless. Therefore, proper setting of stopping criteria can control the effective classification and reduce the possible effect of over-fitting.

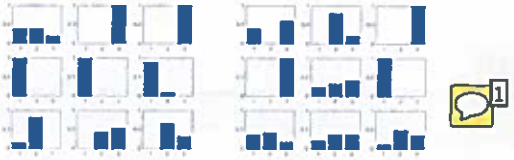
As experiment, we require that the growth stops when a maximum number of depth has been reached or when a node contains less than 5 data points.

When the tree growth is done, the class distributions are stored in the leaf nodes.

Fig.3 shows some visualized leaf nodes when setting the number of trees T=10, the depth of trees D=5, the degree of randomness $\rho = 10$ and various split function as linear, non-linear, axis-aligned and two-pixel test.



(a) linear (b) non-linear



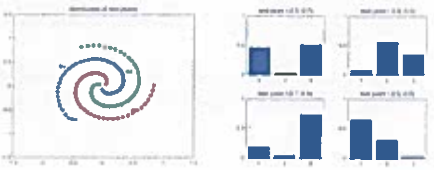
(c) axis-aligned (d) two-pixel

Fig.3 the class distributions on some leaf nodes of the tree with T=10, depth=5, $\rho = 3$ and the split function is

2.2 Evaluating Decision Forest on the test data

Given a previously unseen data point, a decision tree applies hierarchically the split functions stored at each node that the data point passes until reaches a leaf node. This process is repeated in every tree of the forest. Since all trees are trained independently because of the randomness, the decision made by each tree may be different. The forest prediction is finally done by combining all tree predictions such as averaging the distributions or doing majority voting.

Fig.4 presents the classification results of some data points: (-0.5, -0.7), (0.4, 0.3), (-0.7, 0.4), (0.5, -0.5) using the training forest with T=10, D=5, $\rho = 10$ and axis-aligned split function.



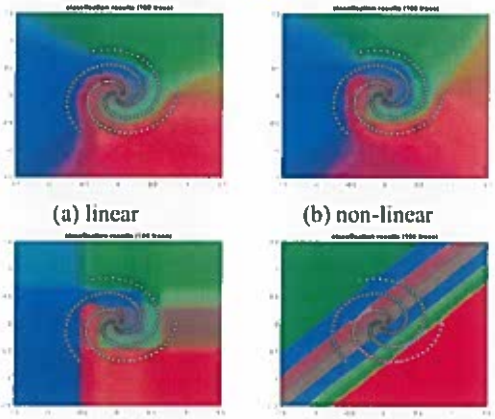
(a) (b)

Fig.4 (a)the distribution of test points with the forest using axis-aligned function and T=10, depth=5, $\rho = 3$. (b) the averaged class distribution of each test points

The forest classifies the point according to the largest probability from the averaged class distribution. The result shows that most of the data points are classified to the nearest class.

The prediction ability of a random forest depends on the training effect that is mainly influenced by the selection of the types of the split function and some parameters such as the number of trees, the depth of trees and the degree of randomness.

(1) The type of spilt function



(a) linear (b) non-linear

(c) axis-aligned (d) two-pixel

Fig.5 the distribution of forest using four split functions with T=100, depth = 5, randomness = 10.

Page: 2

Number: 1 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:44:43
this is very good

Number: 2 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:46:21
Are the histograms tended to be peaky? What does that mean? in terms of entropy/information gain and classification in the end?
Do we discuss/compare different options of stopping?

Number: 3 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:43:30

Number: 4 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:43:38
very good!

Number: 5 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:43:46

Number: 6 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 20:35:46

Number: 7 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 20:36:01
excellent

Number: 8 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 14:44:09

Number: 9 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 14:44:27
what if we use the information gain as a stopping criterion?

Fig.5 illustrates the distribution of forest using four split functions.

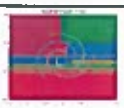
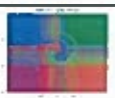
The points in spiral distribution are the training data, while the background shows the classification results of the testing data. The red, green, and blue color donate data with label 1, 2 and 3 respectively, other colors shown in the background represent the uncertain classification.

(2) The number of trees

The Random Forest uses multiple decision trees to predict the unknown data, the average of decisions can approximate the more accurate class distribution.

In a random forest, since the growth of the trees is random, for a given depth, the distributions obtained by trees can be regarded as random samples. The average of samples can approximate the more accurate class distribution than one decision tree.

By observing form tab.2, the larger the number of trees is, the fuzzier the boundary is and the more precise the classification results are. When the number of tree is only 1, the boundary of each class distribution is quite distinct, no matter which split function is utilized. Intuitively, random forest with only one tree generates worse classification results. Since there may be some noise data in the training set, nodes of the only decision tree would compute splitting criteria with these noise data and adversely affect the classification of testing data.

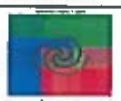
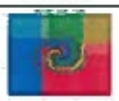
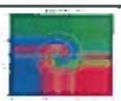
Number of trees	1	10	100
distribution			
training time (s)	0.15	1.55	10.72
testing time (s)	3.81	20.08	285.97

Tab.2 the distribution and some evaluations of testing data with the forest with four functions, parameters of depth=5 randomness=10 and various number of trees.

Although the larger the number of samples is the more accurate the approximation is, considering the computation time and the error-tolerance of the decision, there should be a balance to set the number of the tree.

(3) The depth of trees

The aim of multiple trees is to provide the diversity for the forest, while the depth of the tree influences the diversity as well. The distribution comparing of different depth of the trees is given in tab.3 with parameters tree number=10, randomness=10, using axis-aligned split function.

The depth of trees	5	10	15
distribution			
training time (s)	0.21	0.85	160.16
testing time (s)	16.16	18.76	21.66

Tab.3 the distribution and some evaluations of testing data with the forest with axis-aligned function, parameters of T=10, randomness=10 and various depth of trees.

The distribution with higher depth of trees show better resolution result in the range in tab.3. But the computation time, especially the training time will significantly increase when the depth getting larger and larger.

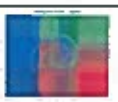
On the other hand, unlike the number of trees that the more trees provide the better diversity, it is even worse if the depth is too large. For example, to the limit that the depth equals to the number of features, every tree would be identical and have no diversity. Therefore, the classification accuracy will have little improvement after the depth get a specific value.

(4) The degree of randomness

The degree of randomness is the parameter that controls the randomness of the forest in training process.

Since when the degree of randomness is set to 1, each split node take only a single randomly chosen set of values for the selection of parameters in its split function, and there is no optimization to compare the better information gain. Therefore, the split is highly random and the decision will to a high probability be undesirable. On the contrary, when it is large enough to contain almost all the possibility, the randomness disappears and trees in forest become highly correlated. It is important to find a balance between randomness and correlation and get a degree of randomness properly that helps to optimize the effect of classification.

An example comparing with various degree of randomness is illustrated in tab.4. (tree number=10, depth=5, using axis-aligned split function).

The degree of randomness	1	20	100
			
training time (s)	0.16	0.90	4.83
testing time (s)	17.91	16.69	16.99

Tab.4 the distribution and some evaluations of testing data with the forest with axis-aligned function, parameters of T=10, depth=5 and various degree of randomness.

Compared with degree 20, colors of testing data in the distribution of degree 1 show more closely aligned. When the degree of randomness increases to 100, the distribution of encoded color does not show spiral shape, and more red encoded testing data is classified to the green encoded class. Thus, only increasing the degree of randomness at the certain range improves the performance of random forest.

3.3 Image categorization

3.3.1 K-means codebook

Given training images, feature detection should be done firstly to describe image patches as descriptor vectors. These descriptors should be scale and affine invariant and rich enough to be discriminative at category level.

K-means clustering is then computed to these descriptors and divides them into K groups in terms of similarity to form the vocabulary.

Page: 3

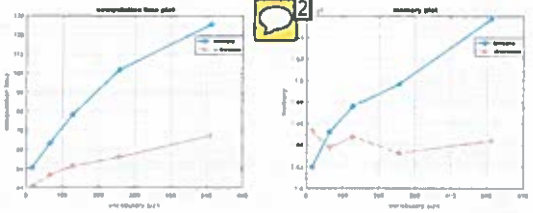
Number: 1 Author: Tae-Kyun Subject: Highlight Date: 23/05/2017 20:36:45

Number: 2 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 20:36:32
wow, beyond the expectations

Number: 3 Author: Tae-Kyun Subject: Sticky Note Date: 23/05/2017 20:37:06
what if we try to observe their combinatorial impacts?

(1) Vocabulary size

The size of the vocabulary (or codebook) depends on the number of clusters that are divided, i.e. the K. Therefore, a vocabulary is made up with K codewords (the centers of clusters). The selection of the K would affect the computation time and the memory consumption of the codebook construction as well as the later data encoding process.



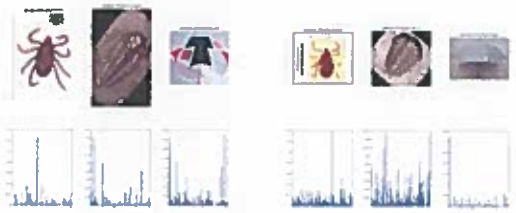
(a) (b)

Fig.6. (a) the computation time of different vocabulary sizes. (b) the memory of different vocabulary sizes.

As illustrated in fig.6, the computation time of codebook construction goes up with the increase of vocabulary size, and the 'vl-k-means' shows better performance in both time efficiency and the memory consumption with size increasing compared to 'k-means'

(2) bag-of-words histograms

An image can be regarded as a collection of visual words. As the vocabulary formed, every visual word is assigned to a nearest codeword, thus a bag of words can be represented as a histogram with K bins which count the number of words assigned to each codeword.



(a) (b)

Fig.7 (a) bag-of-words histograms of some example training images with the vocabulary size of 256. (b) those of testing image.

Fig.7 illustrates some example images and their bow histograms from training and testing sets (10 classes and 15 images of each class). The description method used here is the multi-scaled dense SIFT [2] and 100k descriptors from all are chosen to do K-means clustering and build the visual vocabulary. Descriptors of testing images are detected by the same way and computed the Nearest Neighbor matching with the codebook formed in training stage.

(3) Vector quantization process

Vector quantization is the process encoding descriptor vectors to codewords by some methods like Nearest Neighbor matching.

Visual coding based on K-means vector quantization is effective but time-demanding. A more efficient method is vocabulary trees. The comparing of the computation time of the two methods is given in fig.8.

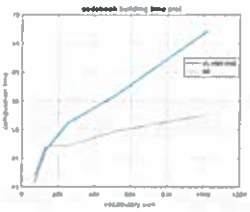


Fig.8 Time efficiency of vector quantization.

3.3.2 RF classifier

As the training and testing bag-of-words with corresponding class labels obtained, the idea of Random Forest can be used as the classifier to do the classification to the testing bag-of-words.

Testing image	Classification	results
	'yin_yang'	Success
	'trilobite'	failure

Tab.5 the example of success and failure.

There are some factors that may affect the recognition accuracy, such as the vocabulary size and the RF parameters.

(1) Vocabulary size

To evaluate the impact of vocabulary size, RF parameters are settled while the size varies. The recognition accuracies with different vocabulary size are given in tab.9.

Vocabulary size	Recognition accuracy (average)	Training time (s)	Testing time (s)
64	64.7%	5.69	0.58
128	62.0%	6.19	0.60
256	64.7%	7.21	0.65
512	69.3%	9.07	0.64
1024	58.0%	12.61	0.70
2048	57.3%	18.31	0.73

Tab.6 Random Forest classification using axis-aligned function, parameters of T=20, depth=10, randomness=10.

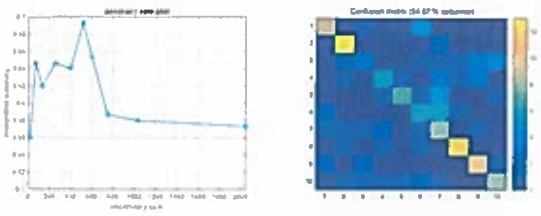


Fig.9 (a) the accuracy trend with change of vocabulary size. (b) the confusion matrix when the size is 512.

The more obvious trends are shown in fig.9 (a), in which the accuracy pick when the vocabulary size is around 500. The possible reason behand is that there are too many unrelated patches being grouped when the size is too small while similar features are more likely to be divided into different clusters when the size is too large.

(2) The degree of randomness

Page: 4

Number: 1 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 19:08:51

Number: 2 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:09:15
this is excellent!

Number: 3 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:10:49
very good, why do we think these succeed or fail?

Number: 4 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:11:31
excellent table

Number: 5 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:12:05
very interesting observation... does the accuracy drop that dramatically?

Number: 6 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 19:09:51

Number: 7 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 19:12:10

Number: 8 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:10:04
excellent

The influence of the degree of randomness on accuracy is shown in fig.10, in which the vocabulary size is settled at 256.

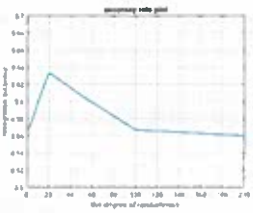
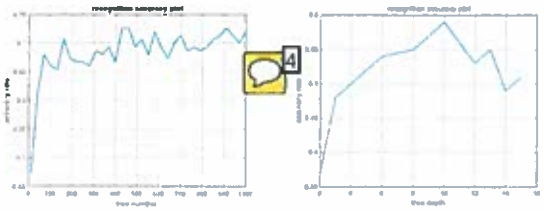


Fig.10 (a) The accuracy of different degree of randomness.

With increase of the degree of randomness, the accuracy rises before 20 where achieves a pick, and then decreases. Because when the degree of randomness is small, the forest shows high randomness. To the limit, when it is set to 1, each split node take only a single randomly chosen set of values for the selection of parameters in its split function, the split is highly random and the decision will to a high probability be undesirable. However, when the degree is too large, the trees in the forest will become high correlated and show high redundancy.

(4) The number of trees

From fig.11 (a), the accuracy rises with the increase of the tree number when there are only a few of trees, which agrees with the fact that the ensemble classifier has better performance than single decision tree. When the number is large enough, the accuracy will not continue rising but fluctuate in range less than 1%. Therefore, a number around 100 can be regarded as the optimal value.



(a) (b)

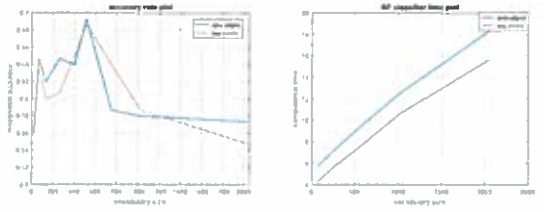
Fig.11 The accuracy of (a) the number of trees, (b) the depth of trees.

(5) The depth of trees

As multiple trees provide the diversity for the forest, the deeper trees may lead to less diversity. Fig.11 (b) shows the accuracy goes up at first and gets a relatively good value when the depth equals 10, then decreases with fluctuation.

(2) The type of spilt function

Fig.12 compares the accuracy and time-efficiency of using axis-aligned and two-pixel function. They accuracy shows the similar trend with the increase of vocabulary size while the axis-aligned function is slightly better than two-pixel considering the time-efficiency.

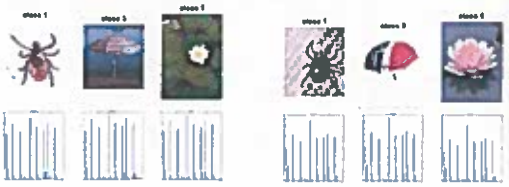


(a) (b)

Fig.12 (a) the accuracy trend with change of vocabulary size. (b) time-efficiency with change of vocabulary size.

3.3.3 RF codebook

RF codebook is the method to use random forest instead of K-means clustering to form the codebook in which the codewords consist of the leaves of the grown forest. The bag-of-words histograms are obtained by accumulating the number of leaves in the same leaf label of each tree.



(a) (b)

Fig.13 (a) bag-of-words histograms of some example training images with the vocabulary size of 256. (b) those of testing image.

To compare the performance of codebook produced by K-means and RF, random forest classifier with specific parameters (axis-aligned function with T=1000, depth=5, $\rho = 3$) is computed to the new bag-of-words with RF codebook.

Testing image	Classification	results
	'yin_yang'	Success
	'umbrella'	failure

Tab.7 the example of success and failure (RF codebook).

(1) Different size of RF codebook

As the codebook is formed by the leaves of the forest and the number of leaves depends on the number and depth of trees, the setting of these two parameters will influence the codebook size and consequently influence the classification results.

The increasing trendency of accuracy and computation time with the rise of tree number can generally be seen in tab.8.

the number of trees	3	5	10	20	50
Codebook size	40	80	152	311	794
Accuracy	68.0%	65.3%	70.7%	74.7%	66.0%
Time of codebook construction	43.94	48.01	56.92	70.99	118.15

Tab.8 the averaged accuracy and time with different numbers of trees and a determined depth= 5 .

The influence given by the depth of trees is in the similar way.

Compared with the utility of K-means codebook, the accuracy and computation time of RF codebook show (in fig.14) similar variation trendency with the change of

Page: 5

Number: 1 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:14:43
In general, this part is excellent...

what do we observe when changing multiple (more than 1) parameters at once? e.g. 3D plot (z: performance, x: parameter 1, y: parameter 2)

Number: 2 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:17:03

Number: 3 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:17:11
what if we tune these parameters?

Number: 4 Author: Tae-Kyun Subject: Sticky Note Date: 24/05/2017 19:12:33
very good

Number: 5 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:16:57
excellent table

Number: 6 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:21:43

Number: 7 Author: Tae-Kyun Subject: Highlight Date: 24/05/2017 19:12:39

vocabulary size that is controled by the number and depth of trees.

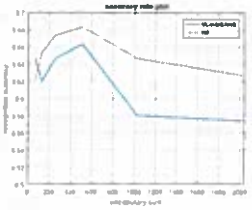


Fig.14 The accuracy with RF and K-means codebook.

The accuracy with the use of RF codebook shows higher than that of K-means at most of the vocabulary size. The reason behind is that the use of K-means codebook relays on the nearest neighbor assignments which is unstable in high dimensional cases. Besides, both kinds of codebooks pick the accuracy at the size around 500.

(2) Vector quantization complexity

K-means has a complexity of $O(DNK)$, where N is the number of patches used and K is the number of codewords and D is the descriptor dimensionality, while RF has around $O(T_{max}NlogK)$, where T_{max} is the number of split function trials. Because the trees in practice are often obtained with good balance, T_{max}~$O(\sqrt{D})$. Therefore, RF has a complexity of $O(\sqrt{D}NlogK)$.

Fig.15 shows complexity in terms of the computation time and memory consumption of both codebooks.

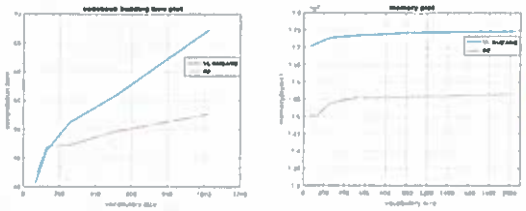


Fig.15 computation time and memory consumption of RF and K-means codebook.

RF codebook always gives better performance in both the efficiency of computation time and memory with the various vocabulary size compared to K-means.

Additionally, the growth of the computation time of K-means codebook construction is linear increase along vocabulary size, while that of RF codebook is seen logarithmically, which just sticks to the theoretical analysis.

4. Improvement in codes

In this experiment, we ameliorate some parts of the provided random forest code, which are not perfect.

Initially, we correct the code of computing the information gain and entropy in function 'splitNode' to obtain the positive value of information gain, instead of negative value before correction.

Next, we make change to the funtion 'updateIG' to update the change in each computation, and visualize the best result of multiple computations, instead of showing the last result as the best result before correction.

The specific codes are shown in appendixII, the codes in appendix marked yellow are the modified parts.

IV. CONCLUSIONS

As a classifier, random forest can naturally handle the multiclass and high dimensional categorization problems and provide probabilistic outputs. With the proper setting of parameters, RF classifier can achieve a relatively high classification accuracy. But the parameters need to be chosen empirically, so there are usually many pre-works to test a set of parameter.

As a coding method, random forest can produce discriminative codebook, and perform better in terms of the accuracy, time-efficiency and memory consumption compared with K-means.

III. REFERENCES

[1] Tea-Kyun Kim. Machine Learning for Computer Vision lecture notes.

[2] http://www.vlfeat.org/matlab/vl_phow.html

Page: 6

Number: 1 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:17:48
this is exceptional!

Number: 2 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:19:36
do we try different parameters in both RF codebook and RF classifier? what are the best parameters in each? do we observe any difference, if yes, why?

Number: 3 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:18:22
is this the time for VQ? or training time?

Number: 4 Author: Tae-Kyun Subject: Highlight Date: 25/05/2017 10:18:07

Number: 5 Author: Tae-Kyun Subject: Sticky Note Date: 25/05/2017 10:18:41
excellent comparisons! on overall

APPENDIX I

The code of building codebook and classifying data:

1. The main code

```
clc;
close all;
clear all;

init;
%% K-means codebook & RF codebook
tic;
[data_train, data_test] = getData('Caltech','RF');
T1=toc;
memory1= memory;
m1=memory1.MemUsedMATLAB;

%% RF classifier

% Set the random forest parameters for instance,
param.num = 10;         % Number of trees
param.depth = 10;        % trees depth
param.splitNum = 10;     % Number of split functions to try
param.split = 'IG';     % Currently support 'information gain' only
param.function = 3;     % 1 -- linear
                    % 2 -- non-linear
                    % 3 -- aixs-aligned
                    % 4 -- two-pixel test
tic
trees=growTrees(data_train,param);
T2=toc;

tic;
p_rf_sum=[];
for n=1:size(data_test,1)
    leaves = testTrees([data_test(n,:) 0],trees,param);
    % average the class distributions of leaf nodes of all trees
    p_rf= trees(1).prob(leaves,:);
    if length(trees)~=1
        p_rf_sum(n,:) = sum(p_rf)/length(trees);
    else
        p_rf_sum(n,:)=p_rf;
    end
end
T3=toc;

[~,c] = max(p_rf_sum');
accuracy_rf = sum(c==data_test(:,end)')/length(c); % Classification accuracy (for Caltech

idx = sub2ind([10, 10], data_test(:,end)', c) ;
conf = zeros(10) ;
conf = vl_binsum(conf, ones(size(idx)), idx) ;
figure;
imagesc(conf) ;
title(sprintf('Confusion matrix (%.2f %% accuracy)', 100 * accuracy_rf) ) ;
```

2. The code of 'getData' function

```
function [ data_train, data_query] = getData( MODE, CODEBOOK)
% Generate training and testing data

% Data Options:
%   1. Toy_Gaussian
%   2. Toy_Spiral
%   3. Toy_Circle
%   4. Caltech 101
```

```
showImg = 0; % Show training & testing images and their image feature vector (histogram
representation)

PHOW_Sizes = [4 8 10]; % Multi-resolution, these values determine the scale of each layer.
PHOW_Step = 8; % The lower the denser. Select from {2,4,8,16}

switch MODE
    case 'Toy_Gaussian' % Gaussian distributed 2D points
        %rand('state', 0);
        %randn('state', 0);
        N= 150;
        D= 2;

        cov1 = randi(4);
        cov2 = randi(4);
        cov3 = randi(4);

        X1 = mgd(N, D, [randi(4)-1 randi(4)-1], [cov1 0;0 cov1]);
        X2 = mgd(N, D, [randi(4)-1 randi(4)-1], [cov2 0;0 cov2]);
        X3 = mgd(N, D, [randi(4)-1 randi(4)-1], [cov3 0;0 cov3]);

        X= real([X1; X2; X3]);
        X= bsxfun(@rdivide, bsxfun(@minus, X, mean(X)), var(X));
        Y= [ones(N, 1); ones(N, 1)*2; ones(N, 1)*3];

        data_train = [X Y];

    case 'Toy_Spiral' % Spiral (from Karpathy's matlab toolbox)

        N= 50;
        t = linspace(0.5, 2*pi, N);
        x = t.*cos(t);
        y = t.*sin(t);

        t = linspace(0.5, 2*pi, N);
        x2 = t.*cos(t+2);
        y2 = t.*sin(t+2);

        t = linspace(0.5, 2*pi, N);
        x3 = t.*cos(t+4);
        y3 = t.*sin(t+4);

        X= [[x' y']; [x2' y2']; [x3' y3']];
        X= bsxfun(@rdivide, bsxfun(@minus, X, mean(X)), var(X));
        Y= [ones(N, 1); ones(N, 1)*2; ones(N, 1)*3];

        data_train = [X Y];

    case 'Toy_Circle' % Circle

        N= 50;
        t = linspace(0, 2*pi, N);
        r = 0.4
        x = r*cos(t);
        y = r*sin(t);

        r = 0.8
        t = linspace(0, 2*pi, N);
        x2 = r*cos(t);
        y2 = r*sin(t);

        r = 1.2;
        t = linspace(0, 2*pi, N);
```

```
        x3 = r*cos(t);
        y3 = r*sin(t);

        X= [[x' y']; [x2' y2']; [x3' y3']];
        Y= [ones(N, 1); ones(N, 1)*2; ones(N, 1)*3];

        data_train = [X Y];

    case 'Caltech' % Caltech dataset
        close all;
        imgSel = [15 15]; % randomly select 15 images each class without replacement. (For
both training & testing)
        folderName = './Caltech_10/101_ObjectCategories';
        classList = dir(folderName);
        classList = {classList(3:end).name} % 10 classes

        disp('Loading training images...')
        % Load Images -> Description (Dense SIFT)
        cnt = 1;
        if showImg
            figure('Units','normalized','Position',[.05 .1 .4 .9]);
            suptitle('Training image samples');
        end
        for c = 1:length(classList)
            subFolderName = fullfile(folderName,classList{c});
            imgList = dir(fullfile(subFolderName,'*.jpg'));
            imgIdx{c} = randperm(length(imgList));
            imgIdx_tr = imgIdx{c}(1:imgSel(1));
            imgIdx_te = imgIdx{c}(imgSel(1)+1:sum(imgSel));

            for i = 1:length(imgIdx_tr)
                I = imread(fullfile(subFolderName,imgList(imgIdx_tr(i)).name));

                % Visualise
                if i < 6 & showImg
                    subaxis(length(classList),5,cnt,'SpacingVert',0,'MR',0);
                    imshow(I);
                    cnt = cnt+1;
                    drawnow;
                end

                if size(I,3) == 3
                    I = rgb2gray(I); % PHOW work on gray scale image
                end

                % For details of image description, see
http://www.vlfeat.org/matlab/vl_phow.html
                [~, desc_tr{c,i}] = vl_phow(single(I),'Sizes',PHOW_Sizes,'Step',PHOW_Step); %
extracts PHOW features (multi-scaled Dense SIFT)
            end
        end

        disp('Building visual codebook...')
        % Build visual vocabulary (codebook) for 'Bag-of-Words method'
        desc_sel_tr = single(vl_colsubset(cat(2,desc_tr{:}), 10e4)); % Randomly select 100k
SIFT descriptors for clustering
        switch CODEBOOK

            case 'VL_KMEANS'
% K-means clustering
        numBins = 2048; % for instance,

        % the bag-of-words histograms (training data)
        figure;
```

```
[C,ID]=vl_kmeans(desc_sel_tr,numBins); % VL-KMEANS
for i=1:1:10
    for j=1:1: 15
        d=double(desc_tr{i,j}.');
        d2=d.^2;
        d2=sum(d2,2);
        d2=repmat(d2,1,size(C,2)); % VL-KMEANS
        C2=C.^2;% VL-KMEANS
        C2=sum(C2);
        C2=repmat(C2,size(d2,1),1);
        dis=d2+C2-2.*(d*C);% VL-KMEANS
        [value, index]=min(dis, [], 2);
        his_tr=zeros(1,numBins);
        for n=1:1:length(index)
            his_tr(index(n))=his_tr(index(n))+1;
        end
        encode_tr{i,j}=his_tr;
        if j<=4
            subplot(10,4,(i-1)*4+j);
            bar(his_tr'/size(desc_tr{i,j},2));
        end
    end
end
suptitle('bag-of-words histograms of example training images');
% training data
data_train=zeros(10*15,numBins+1);
for i=1:10
    for j=1:15
        data_train((i-1)*15+j,:)=[encode_tr{i,j} i];
    end
end
case 'KMEANS'
numBins = 512; % for instance,

% the bag-of-words histograms (training data)
figure;
desc_sel_tr=desc_sel_tr.';
[ID,C]=kmeans(desc_sel_tr,numBins,'Distance','sqeuclidean','Replicates',1);
for i=1:1:10
    for j=1:1:15
        d=double(desc_tr{i,j}.');
        d2=d.^2;
        d2=sum(d2,2);
        d2=repmat(d2,1,size(C,1));  % KMEANS
        C2=C'.^2;% KMEANS
        C2=sum(C2);
        C2=repmat(C2,size(d2,1),1);
        dis=d2+C2-2.*(d*C');% KMEANS
        [value, index]=min(dis, [], 2);
        his_tr=zeros(1,numBins);
        for n=1:1:length(index)
            his_tr(index(n))=his_tr(index(n))+1;
        end
        encode_tr{i,j}=his_tr;
        if j<=4
            subplot(10,4,(i-1)*4+j);
            bar(his_tr'/size(desc_tr{i,j},2));
        end
    end
end
suptitle('bag-of-words histograms of example training images');
% training data
data_train=zeros(10*15,numBins+1);
for i=1:10
    for j=1:15
        data_train((i-1)*15+j,:)=[encode_tr{i,j} i];
```

```
            end
        end

        case 'RF'
            % RF
            % Set the random forest parameters for instance,
            param.num = 4;          % Number of trees
            param.depth = 6;        % trees depth
            param.splitNum = 10;     % Number of split functions to try
            param.split = 'IG';     % Currently support 'information gain' only
            param.function = 3;     % 1 -- linear
                                    % 2 -- non-linear
                                    % 3 -- aixs-aligned
                                    % 4 -- two-pixel test
            desc_sel_tr=[desc_sel_tr;ones(1,100000)];
            trees=growTrees(desc_sel_tr.',param);

            p_rf_sum=[];
            for i=1:1:10
                for j=1:1:15
                    d=double(desc_tr{i,j}.');
                    d=[d ones(size(desc_tr{i,j},2),1)];

                    leaves = testTrees_fast(d,trees);
                    leaves(~leaves)=1;
                    his_tr=zeros(1,size(trees(1).prob,1));
                    for a=1:1:size(leaves,1)
                        for b=1:1:param.num
                            his_tr(leaves(a,b))=his_tr(leaves(a,b))+1;
                        end
                    end

                    encode_tr{i,j}=his_tr;
                    if j<=4
                        subplot(10,4,(i-1)*4+j);
                        bar(his_tr'/size(desc_tr{i,j},2));
                    end
                % average the class distributions of leaf nodes of all trees
                end
            end
                    suptitle('bag-of-words histograms of example training images');

        disp('Encoding Images...')

    % training data
        data_train=zeros(10*15,size(trees(1).prob,1)+1);
        for i=1:10
            for j=1:15
                data_train((i-1)*15+j,:)=[encode_tr{i,j} i];
            end
        end
        end

        % Clear unused varibles to save memory
        clearvars desc_tr desc_sel
    end
    switch MODE
        case 'Caltech'
            if showImg
            figure('Units','normalized','Position',[.05 .1 .4 .9]);
            suptitle('Testing image samples');
            end
            disp('Processing testing images...');

            cnt = 1;
```

```
        % Load Images -> Description (Dense SIFT)
        for c = 1:length(classList)
            subFolderName = fullfile(folderName,classList{c});
            imgList = dir(fullfile(subFolderName,'*.jpg'));
            imgIdx_te = imgIdx{c}(imgSel(1)+1:sum(imgSel));

            for i = 1:length(imgIdx_te)
                I = imread(fullfile(subFolderName,imgList(imgIdx_te(i)).name));

                % Visualise
                if i < 6 & showImg
                    subaxis(length(classList),5,cnt,'SpacingVert',0,'MR',0);
                    imshow(I);
                    cnt = cnt+1;
                    drawnow;
                end

                if size(I,3) == 3
                    I = rgb2gray(I);
                end
                [~, desc_te{c,i}] = vl_phow(single(I),'Sizes',PHOW_Sizes,'Step',PHOW_Step);
            end
        end
        switch CODEBOOK
            case 'VL_KMEANS'
% the bag-of-words histograms (testing data)
        figure;

        for i=1:1:10
            for j=1:1:15
                d=double(desc_te{i,j}.');
                d2=d.^2;
                d2=sum(d2,2);
                d2=repmat(d2,1,size(C,2)); % VL-kMEANS
                C2=C.^2;% VL-kMEANS
                C2=sum(C2);
                C2=repmat(C2,size(d2,1),1);
                dis=d2+C2-2.*(d*C); % VL-kMEANS
                [value, index]=min(dis, [], 2);
                his_te=zeros(1,numBins);
                for n=1:1:length(index)
                    his_te(index(n))=his_te(index(n))+1;
                end
                encode_te{i,j}=his_te;
                if j<=4
                    subplot(10,4,(i-1)*4+j);
                    bar(his_te'/size(desc_te{i,j},2));
                end
            end
        end
        suptitle('bag-of-words histograms of example testing images');
        % testing data
        data_query=zeros(10*15,numBins+1);
        for i=1:10
            for j=1:15
                data_query((i-1)*15+j,:)=[encode_te{i,j} i];
            end
        end
        case 'KMEANS'
                figure;
        for i=1:1:10
            for j=1:1:15
                d=double(desc_te{i,j}.');
                d2=d.^2;
                d2=sum(d2,2);
                d2=repmat(d2,1,size(C,1)); % kMEANS
```

```
            C2=C'.^2; % kMEANS
            C2=sum(C2);
            C2=repmat(C2,size(d2,1),1);
            dis=d2+C2-2.*(d*C'); % kMEANS
            [value, index]=min(dis, [], 2);
            his_te=zeros(1,numBins);
            for n=1:1:length(index)
               his_te(index(n))=his_te(index(n))+1;
            end
            encode_te{i,j}=his_te;
            if j<=4
               subplot(10,4,(i-1)*4+j);
               bar(his_te'/size(desc_te{i,j},2));
            end
         end
      end
      suptitle('bag-of-words histograms of example testing images');
% testing data
      data_query=zeros(10*15,numBins+1);
      for i=1:10
         for j=1:15
            data_query((i-1)*15+j,:)=[encode_te{i,j} i];
         end
      end
         case 'RF'
            p_rf_sum=[];
            figure;
            for i=1:1:10
               for j=1:1:15
                  d=double(desc_te{i,j}.');
                  d=[d ones(size(desc_te{i,j},2),1)];
                  leaves = testTrees_fast(d,trees);
                  leaves(~leaves)=1;
                  his_te=zeros(1,size(trees(1).prob,1));
                  for a=1:1:size(leaves,1)
                     for b=1:1:param.num
                        his_te(leaves(a,b))=his_te(leaves(a,b))+1;
                     end
                  end

                  encode_te{i,j}=his_te;
                  if j<=4
                     subplot(10,4,(i-1)*4+j);
                     bar(his_te'/size(desc_te{i,j},2));
                  end
            % average the class distributions of leaf nodes of all trees
               end
            end
            suptitle('bag-of-words histograms of example testing images');

% testing data
      data_query=zeros(10*15,size(trees(1).prob,1)+1);
      for i=1:10
         for j=1:15
            data_query((i-1)*15+j,:)=[encode_te{i,j} i];
         end
      end
      end

%       suptitle('Testing image samples');
            if showImg
         figure('Units','normalized','Position',[.5 .1 .4 .9]);
      suptitle('Testing image representations: 256-D histograms');
            end
   otherwise % Dense point for 2D toy data
      xrange = [-1.5 1.5];
```

```
        yrange = [-1.5 1.5];
        inc = 0.02;
        [x, y] = meshgrid(xrange(1):inc:xrange(2), yrange(1):inc:yrange(2));
        data_query = [x(:) y(:) zeros(length(x)^2,1)];
        end
end
```

3. The code of 'testTrees' function

```
function label = testTrees(data,tree,param)
% Slow version - pass data point one-by-one
cc = [];
for T = 1:length(tree)
    for m = 1:size(data,1);
        idx = 1;

        while tree(T).node(idx).dim
            t = tree(T).node(idx).t;
            dim = tree(T).node(idx).dim;

            % Decision
            if param.function == 1 % linear
                if data(m,t(3)) < t(1)*data(m,dim)+ t(2)% Pass data to left node
                    idx = idx*2;
                else
                    idx = idx*2+1; % and to right
                end
            elseif param.function == 2 % non-linear
                if data(m,t(3)) < t(1)*data(m,dim).^2+ t(2)% Pass data to left node
                    idx = idx*2;
                else
                    idx = idx*2+1; % and to right
                end
            elseif param.function == 3    % axis-aligned
                if data(m,dim) < t % Pass data to left node
                    idx = idx*2;
                else
                    idx = idx*2+1; % and to right
                end
            else % two-pixel test
                if data(m,dim)-data(m,t(2)) < t(1) % Pass data to left node
                    idx = idx*2;
                else
                    idx = idx*2+1; % and to right
                end
            end

            dim=tree(T).node(idx).dim;
            if dim==0
                if(isempty(tree(T).node(idx).leaf_idx))
                    idx=floor(idx/2)*4+1-idx;
                end
            end
        end

        leaf_idx = tree(T).node(idx).leaf_idx;
        if ~isempty(tree(T).leaf(leaf_idx))
            p(m,:,T) = tree(T).leaf(leaf_idx).prob;
            label(m,T) = tree(T).leaf(leaf_idx).label;

        end
    end
end

end
```

APPENDIX II

The correction of provided random forest code:

```
function [node,nodeL,nodeR,idx_] = splitNode(data,node,param)
% Split node

visualise =0;

% Initilise child nodes
iter = param.splitNum;
nodeL = struct('idx',[],'t',nan,'dim',0,'prob',[]);
nodeR = struct('idx',[],'t',nan,'dim',0,'prob',[]);

if length(node.idx) <= 5 % make this node a leaf if has less than 5 data points
    node.t = nan;
    node.dim = 0;
    return;
end

idx = node.idx;
data = data(idx,:);
[N,D] = size(data);
ig_best = -inf; % Initialise best information gain
idx_best = [];
for n = 1:iter

    % Split function - Modify here and try other types of split function
    if param.function == 1 % linear
        dim = randi(D-1); % Pick one random dimension
        dim2=dim;
        while dim2==dim
            dim2=randi(D-1);
        end
            k=tan((rand-0.5)*180);
            b_min = single(min(data(:,dim2)-k*data(:,dim))) + eps; % Find the data range of
this dimension
            b_max = single(max(data(:,dim2)-k*data(:,dim))) - eps;
            b=random('unif',b_min,b_max);
            th=k*data(:,dim)+b;
            idx_ = data(:,dim2) < th;
            t=[k b dim2];

    elseif param.function == 2  % non-linear
        dim = randi(D-1); % Pick one random dimension
        dim2=dim;
        while dim2==dim
            dim2=randi(D-1);
        end

        a=2*rand-1;
        b=(data(:,dim2))-a*data(:,dim).^2;
        b_max = single(max(b)) - eps;
        b_min=single(min(b)) + eps;
        b0=random('unif',b_min,b_max);
        idx_ = data(:,dim2) < a*data(:,dim).^2+b0;
        t=[a b0 dim2];

    elseif param.function == 3    % axis-aligned
        dim = randi(D-1); % Pick one random dimension
        d_min = single(min(data(:,dim))) + eps; % Find the data range of this dimension
        d_max = single(max(data(:,dim))) - eps;
        t = d_min + rand*((d_max-d_min)); % Pick a random value within the range as threshold
        idx_ = data(:,dim) < t;

    else % two-pixel test
```

```
        dim = randi(D-1); % Pick one random dimension
        dim2=dim;
        while dim2==dim
            dim2 = randi(D-1); % Pick one random dimension
        end
        d_min = single(min(data(:,dim)-data(:,dim2))) + eps; % Find the data range of this
dimension
        d_max = single(max(data(:,dim)-data(:,dim2))) - eps;
        th = d_min + rand*((d_max-d_min)); % Pick a random value within the range as threshold
        idx_ = data(:,dim)-data(:,dim2) < th;
        t=[th dim2];
    end

% % % %
```